

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	7
1.2 Описание выходных данных.....	7
2 МЕТОД РЕШЕНИЯ.....	9
3 ОПИСАНИЕ АЛГОРИТМОВ.....	12
3.1 Алгоритм метода setNetworkSpeed класса NetworkDevice.....	12
3.2 Алгоритм метода getNetworkSpeed класса NetworkDevice.....	12
3.3 Алгоритм метода setMemoryCapacity класса StorageDevice.....	13
3.4 Алгоритм метода getMemoryCapacity класса StorageDevice.....	13
3.5 Алгоритм конструктора класса NetworkDevice.....	13
3.6 Алгоритм конструктора класса StorageDevice.....	14
3.7 Алгоритм метода printInfo класса NetworkStorage.....	14
3.8 Алгоритм конструктора класса NetworkStorage.....	15
3.9 Алгоритм функции main.....	15
3.10 Алгоритм функции printDevices.....	17
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	19
5 КОД ПРОГРАММЫ.....	26
5.1 Файл main.cpp.....	26
5.2 Файл NetworkDevice.cpp.....	27
5.3 Файл NetworkDevice.h.....	28
5.4 Файл NetworkStorage.cpp.....	28
5.5 Файл NetworkStorage.h.....	28
5.6 Файл StorageDevice.cpp.....	29
5.7 Файл StorageDevice.h.....	29
6 ТЕСТИРОВАНИЕ.....	30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	31

1 ПОСТАНОВКА ЗАДАЧИ

В системе управления цифровыми устройствами используются два типа объектов. Первый тип описывает сетевые устройства, обладающие параметром скорости соединения. Второй тип описывает устройства хранения данных, обладающие объемом памяти.

Некоторые устройства объединяют свойства обоих типов. Такие устройства называются сетевыми хранилищами. Они могут одновременно работать в сети и хранить данные. Для моделирования такой системы используются:

- класс сетевого устройства;
- класс устройства хранения данных;
- класс сетевого хранилища, который наследует свойства обоих классов.

Требуется разработать систему, которая

- демонстрирует использование множественного наследования;
- демонстрирует создание объектов производного класса;
- демонстрирует хранение указателей на объекты в контейнере vector;
- демонстрирует обработку нулевого указателя;
- демонстрирует изменение объектов контейнера по заданному условию.

Класс сетевого устройства

- В закрытом разделе:
 - целочисленное свойство скорости сети.

В открытом разделе

- параметризованный конструктор;
- метод изменения скорости;
- метод получения скорости.

Класс устройства хранения данных

В закрытом разделе

- целочисленное свойство объёма памяти.

В открытом разделе

- параметризованный конструктор;
- метод изменения объёма памяти;
- метод получения объёма памяти.

Класс сетевого хранилища должен:

- наследоваться от двух базовых классов;
- содержать строковое свойство имени устройства;
- иметь конструктор с параметрами:
 - о имя устройства;
 - о скорость сети;
 - о объём памяти;
- иметь метод вывода информации об объекте. Метод вывода должен

отображать: <имя> <скорость> <память>

Алгоритм работы программы

1. Ввести количество устройств.
2. Объявить контейнер указателей на объекты сетевого хранилища.
3. Для каждого устройства ввести: <имя> <скорость> <память>, динамически создать объект сетевого хранилища и сохранить указатель на него в контейнере.
4. После ввода всех устройств добавить в контейнер один нулевой указатель.
5. Вывести строку Initial state
6. Вывести информацию обо всех элементах контейнера:
 - 6.1. если указатель не равен nullptr, вывести информацию об

устройстве.

6.2. если указатель равен, вывести Empty pointer.

7. Ввести значение минимально допустимой скорости сети.

8. Для каждого элемента контейнера:

8.1. если указатель равен nullptr, перейти к следующему элементу;

8.2. если скорость устройства меньше введённого значения:

8.2.1. удалить старый объект;

8.2.2. создать новый объект с тем же именем и объёмом памяти;

8.2.3. установить скорость сети равной $2 * \text{введённое значение}$;

8.2.4. сохранить указатель на новый объект в контейнере.

9. Вывести строку Modified state

10. Вывести информацию обо всех элементах контейнера в том же формате.

11. Освободить память, удалив все динамически созданные объекты.

1.1 Описание входных данных

```
3
NAS1 100 500
NAS2 40 1000
NAS3 150 750
80
```

1.2 Описание выходных данных

```
Initial state
NAS1 100 500
NAS2 40 1000
```

NAS3 150 750
Empty pointer
Modified state
NAS1 100 500
NAS2 160 1000
NAS3 150 750
Empty pointer

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `devices` класса `vector<NetworkStorage*>` предназначен для хранения указателей на объекты `NetworkStorage` для демонстрации полиморфизма и работы с иерархией наследования и управления коллекцией сетевых хранилищ;
- функция `printDevices` для вывода информации обо всех элементах контейнера;
- функция `main` для организации работы программы: ввода кол-ва устр-в, создания контейнера, добавления объектов, вывода состояния, изменения объектов по условию, освобождения памяти;
- `vector` - STL класс, предназначенный для хранения последовательности элементов одного типа;
- множественное наследование - наследование свойств и методов нескольких базовых классов.

Класс `NetworkDevice`:

- свойства/поля:
 - поле целочисленное свойство скорости сети:
 - наименование — `networkSpeed`;
 - тип — цел;
 - модификатор доступа — `private`;
- функционал:
 - метод `NetworkDevice` — параметризованный конструктор для инициализации скорости сети;
 - метод `setNetworkSpeed` — метод для изменения скорости сети;
 - метод `getNetworkSpeed` — метод для получения скорости сети.

Класс StorageDevice:

- свойства/поля:
 - поле целочисленное свойство объёма памяти:
 - наименование — memoryCapacity;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - метод StorageDevice — параметризованный конструктор для инициализации объёма памяти;
 - метод setMemoryCapacity — метод изменения объёма памяти;
 - метод getMemoryCapacity — метод получения объёма памяти.

Класс NetworkStorage:

- свойства/поля:
 - поле строковое свойство имени устройства:
 - наименование — name;
 - тип — строка;
 - модификатор доступа — public;
- функционал:
 - метод printInfo — метод вывода информации об объекте в формате <имя> <скорость> <память>;
 - метод NetworkStorage — конструктор для инициализации имени устройства скорости сети и объёма памяти.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	NetworkDevice			класс сетевого устройства, содержащий закрытое свойство скорости сети и методы для управления им	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
		NetworkStorage	public		3
2	StorageDevice			класс устройства хранения данных, содержащий закрытое свойство объема памяти и методы для управления им	
		NetworkStorage	public		3
3	NetworkStorage			класс сетевого хранилища, который наследует свойства обоих базовых классов классов через множественное наследование, объединяя их возможности с добавлением строкового имени	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `setNetworkSpeed` класса `NetworkDevice`

Функционал: метод изменения скорости сети.

Параметры: цел `speed` - новое значение скорости сети.

Возвращаемое значение: метод не возвращает значения.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `setNetworkSpeed` класса `NetworkDevice`

№	Предикат	Действия	№ перехода
1		присвоить полю <code>networkSpeed</code> знач. параметра <code>speed</code>	Ø

3.2 Алгоритм метода `getNetworkSpeed` класса `NetworkDevice`

Функционал: метод получения скорости сети.

Параметры: нет.

Возвращаемое значение: цел - текущее значение скорости сети.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода `getNetworkSpeed` класса `NetworkDevice`

№	Предикат	Действия	№ перехода
1		вернуть знач. <code>networkSpeed</code>	Ø

3.3 Алгоритм метода setMemoryCapacity класса StorageDevice

Функционал: метод изменения объёма памяти.

Параметры: цел capacity - новое значение объёма памяти.

Возвращаемое значение: метод не возвращает значения.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода setMemoryCapacity класса StorageDevice

№	Предикат	Действия	№ перехода
1		присвоить полю memoryCapacity знач. параметра capacity	Ø

3.4 Алгоритм метода getMemoryCapacity класса StorageDevice

Функционал: метод получения объёма памяти.

Параметры: нет.

Возвращаемое значение: цел - текущее значение объёма памяти.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода getMemoryCapacity класса StorageDevice

№	Предикат	Действия	№ перехода
1		вернуть знач. memoryCapacity	Ø

3.5 Алгоритм конструктора класса NetworkDevice

Функционал: параметризированный конструктор для инициализации скорости сети.

Параметры: цел speed - начальное значение скорости сети.

Алгоритм конструктора представлен в таблице 6.

Таблица 6 – Алгоритм конструктора класса *NetworkDevice*

№	Предикат	Действия	№ перехода
1		иниц. поля <code>networkSpeed</code> значением параметра <code>speed</code>	Ø

3.6 Алгоритм конструктора класса *StorageDevice*

Функционал: параметризированный конструктор для инициализации объёма памяти.

Параметры: цел `capacity` - начальное значение объёма памяти.

Алгоритм конструктора представлен в таблице 7.

Таблица 7 – Алгоритм конструктора класса *StorageDevice*

№	Предикат	Действия	№ перехода
1		иниц. поля <code>memoryCapacity</code> значением параметра <code>capacity</code>	Ø

3.7 Алгоритм метода `printInfo` класса *NetworkStorage*

Функционал: метод вывода информации об объекте в формате `<имя>` `<скорость>` `<память>`.

Параметры: нет.

Возвращаемое значение: метод не возвращает значения.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода `printInfo` класса *NetworkStorage*

№	Предикат	Действия	№ перехода
1		вывести на консоль <code>name</code> , " ", <code>getNetworkSpeed()</code> , " ", <code>getMemoryCapacity()</code> , <code>endl</code>	Ø

3.8 Алгоритм конструктора класса NetworkStorage

Функционал: конструктор для инициализации имени устройства скорости сети и объёма памяти.

Параметры: строк. name - имя устройства, цел speed - скорости сети, цел capacity - объём памяти.

Алгоритм конструктора представлен в таблице 9.

Таблица 9 – Алгоритм конструктора класса NetworkStorage

№	Предикат	Действия	№ перехода
1		вызов конструктора NetworkDevice с парам. speed	2
2		вызов конструктора StorageDevice с парам. capacity	3
3		иниц. поля name со знач. name	Ø

3.9 Алгоритм функции main

Функционал: организация работы программы: ввод кол-ва устр-в, создание контейнера, добавление объектов, вывод состояния, изменение объектов по условию, освобождение памяти.

Параметры: нет.

Возвращаемое значение: цел - код возврата программы.

Алгоритм функции представлен в таблице 10.

Таблица 10 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		объявл. цел deviceCount	2
2		ввод знач. deviceCount	3
3		объявл. контейнера devices типа vector<NetworkStorage*>	4
4		иниц. счётчика i = 0	5

№	Предикат	Действия	№ перехода
5	$i < \text{deviceCount}$	объявл. строк. перем. name	6
			10
6		объявл. цел. перем. speed и capacity	7
7		ввод знач. name, speed, capacity	8
8		создание динамич. объекта NetworkStorage с парам. name, speed, capacity и добавление указателя на него в контейнер devices через push_back	9
9		инкремент i	10
10		добавление nullptr в контейнер devices	11
11		вывод на консоль "Initial state"	12
12		вызов функции printDevices с парам. devices	13
13		объявл. цел. перем. minSpeed	14
14		ввод знач. minSpeed	15
15		иниц. счётчика $i = 0$	16
16	$i < \text{deviceCount}$		17
			25
17	$\text{devices}[i] == \text{nullptr}$		24
			18
18	$\text{devices}[i] \rightarrow \text{getNetworkSpeed()} < \text{minSpeed}$	объявл. строк. перем. name	19
			24
19		объявл. цел. перем. capacity	20
20		присвоить name значение $\text{devices}[i] \rightarrow \text{name}$	21
21		присвоить capacity значение $\text{devices}[i] \rightarrow \text{getMemoryCapacity()}$	22
22		удаление старого объекта, на который указывает $\text{devices}[i]$	23

№	Предикат	Действия	№ перехода
23		создание нового объекта NetworkStorage с парам. name, результат операции minSpeed * 2, capacity и сохранение указателя на него в devices[i]	24
24		инкремент i	16
25		вывод на консоль "Modified state"	26
26		вызов функции printDevices с парам. devices	27
27		иниц. счётчика i = 0	28
28	i < deviceCount		29
			31
29	devices[i] != nullptr	удаление объекта, на который указывает devices[i]	30
			30
30		инкремент i	28
31		возврат 0	Ø

3.10 Алгоритм функции printDevices

Функционал: для вывода информации обо всех элементах контейнера.

Параметры: vector<NetworkStorage*>& devices - ссылка на контейнер указателей на объекты NetworkStorage.

Возвращаемое значение: функция не возвращает значения.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции printDevices

№	Предикат	Действия	№ перехода
1		иниц. счётчика i = 0	2
2	i < devices.size()		3
			5
3	devices[i] != nullptr	вызов devices[i]->printInfo()	4
		вывести на консоль "Empty pointer", endl	4

№	Предикат	Действия	№ перехода
4		инкремент i	2
5		завершение цикла	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-7.

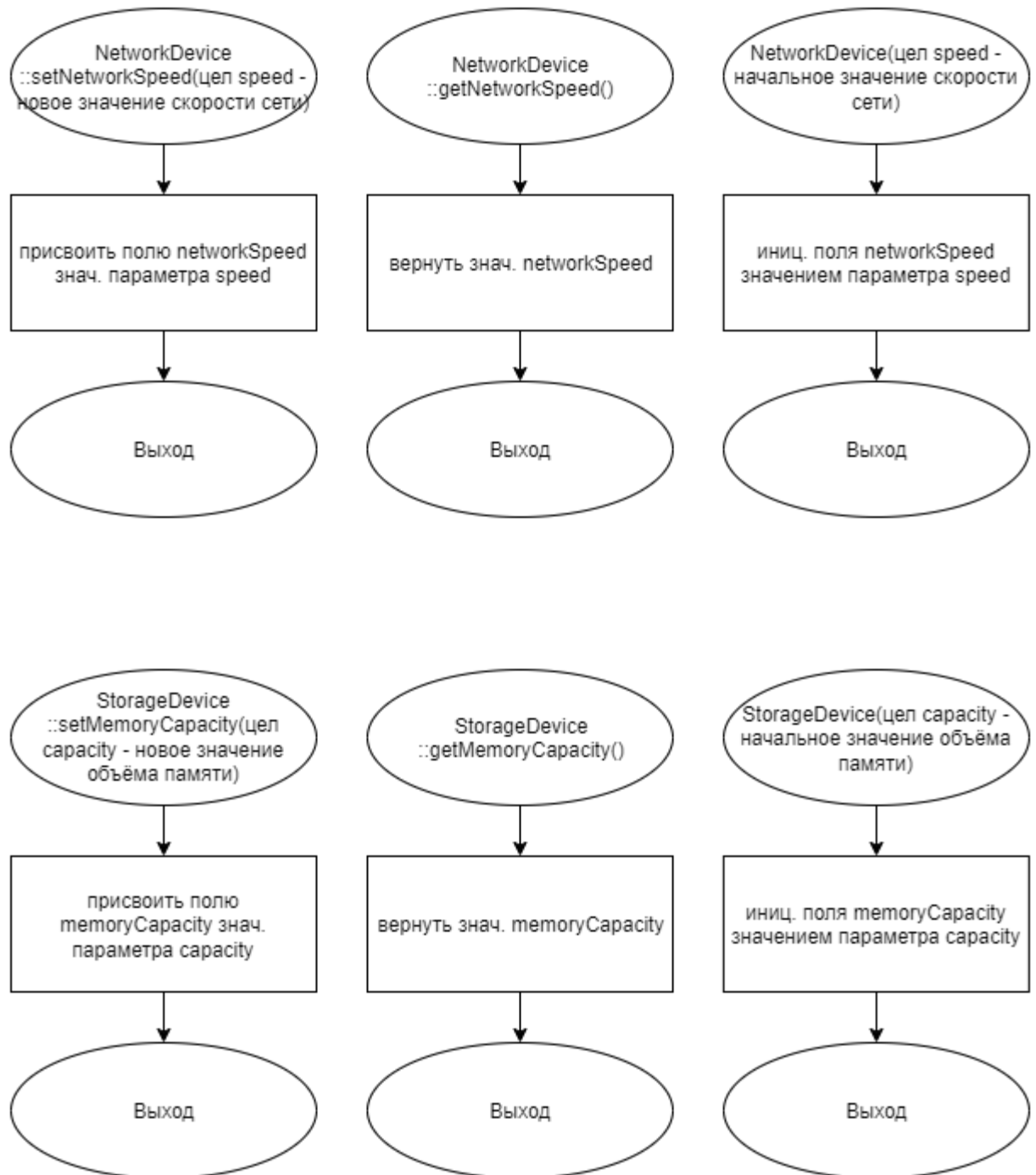


Рисунок 1 – Блок-схема алгоритма

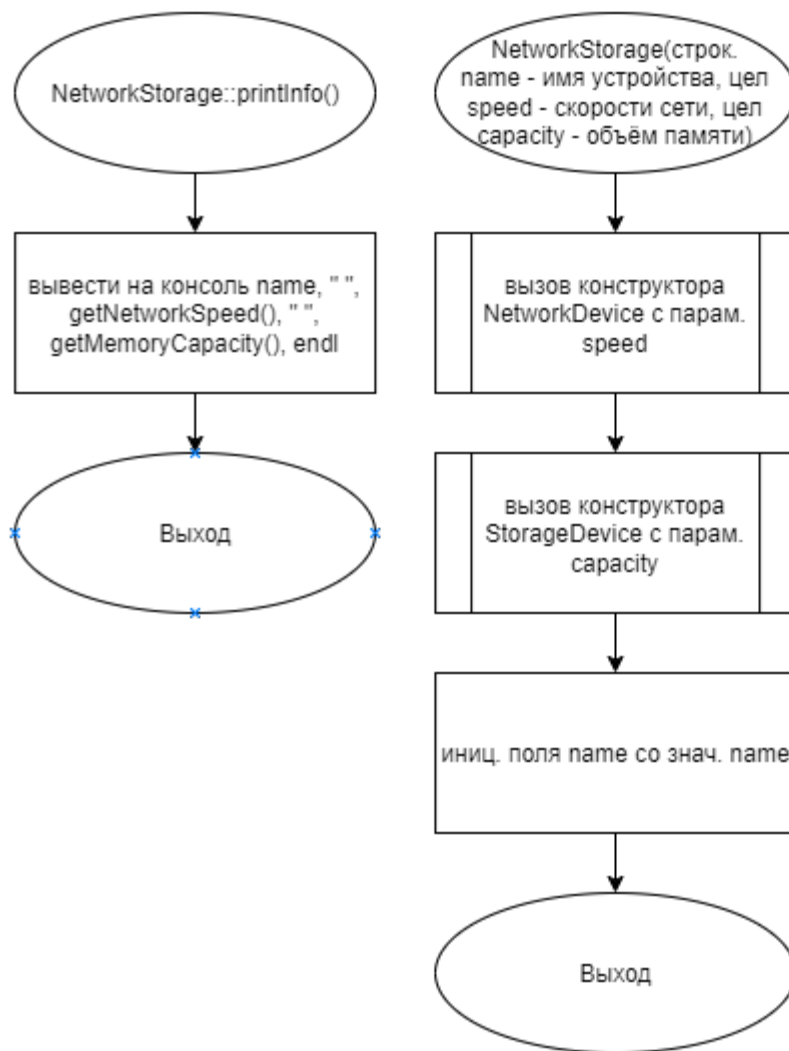


Рисунок 2 – Блок-схема алгоритма

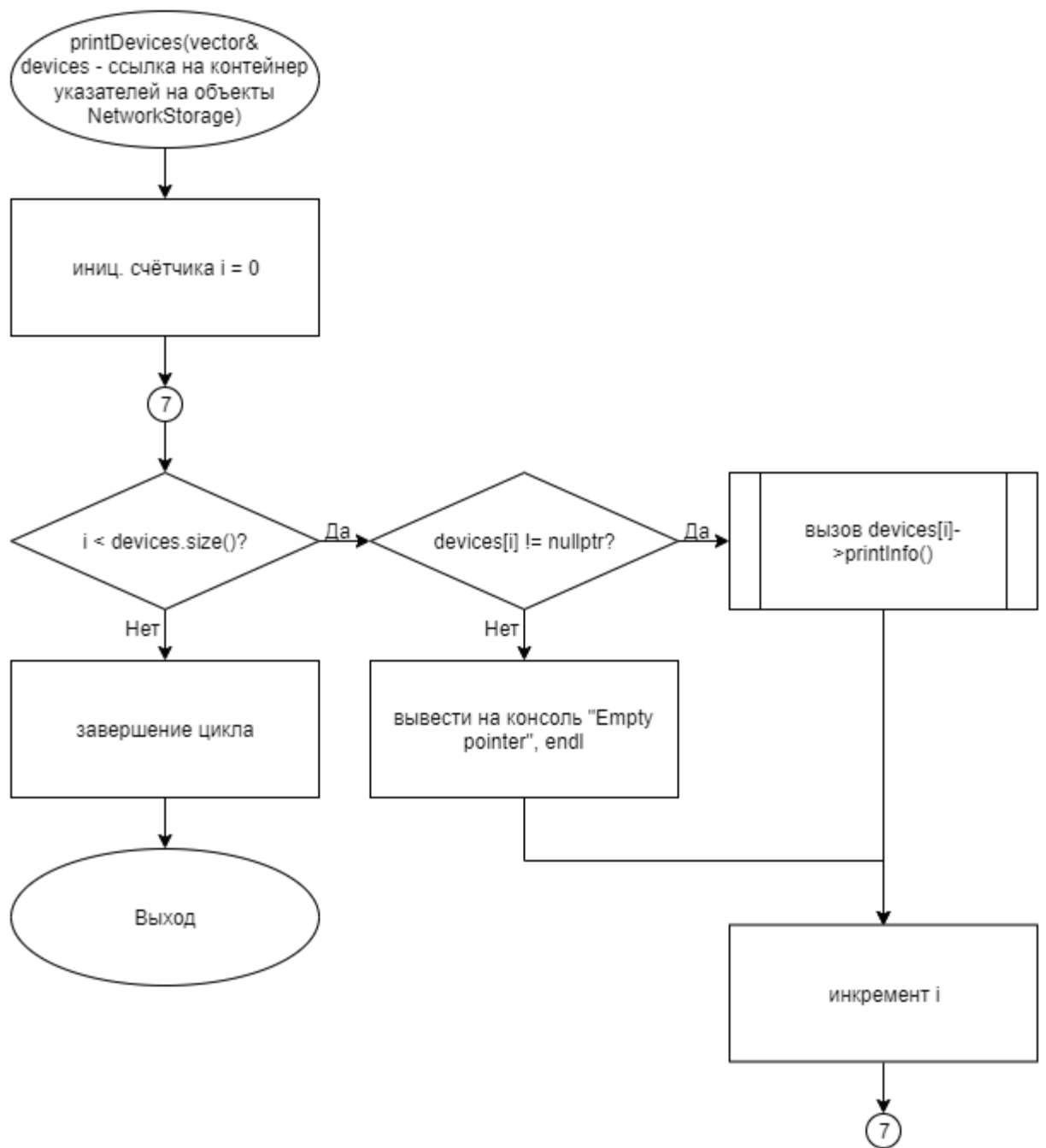


Рисунок 3 – Блок-схема алгоритма

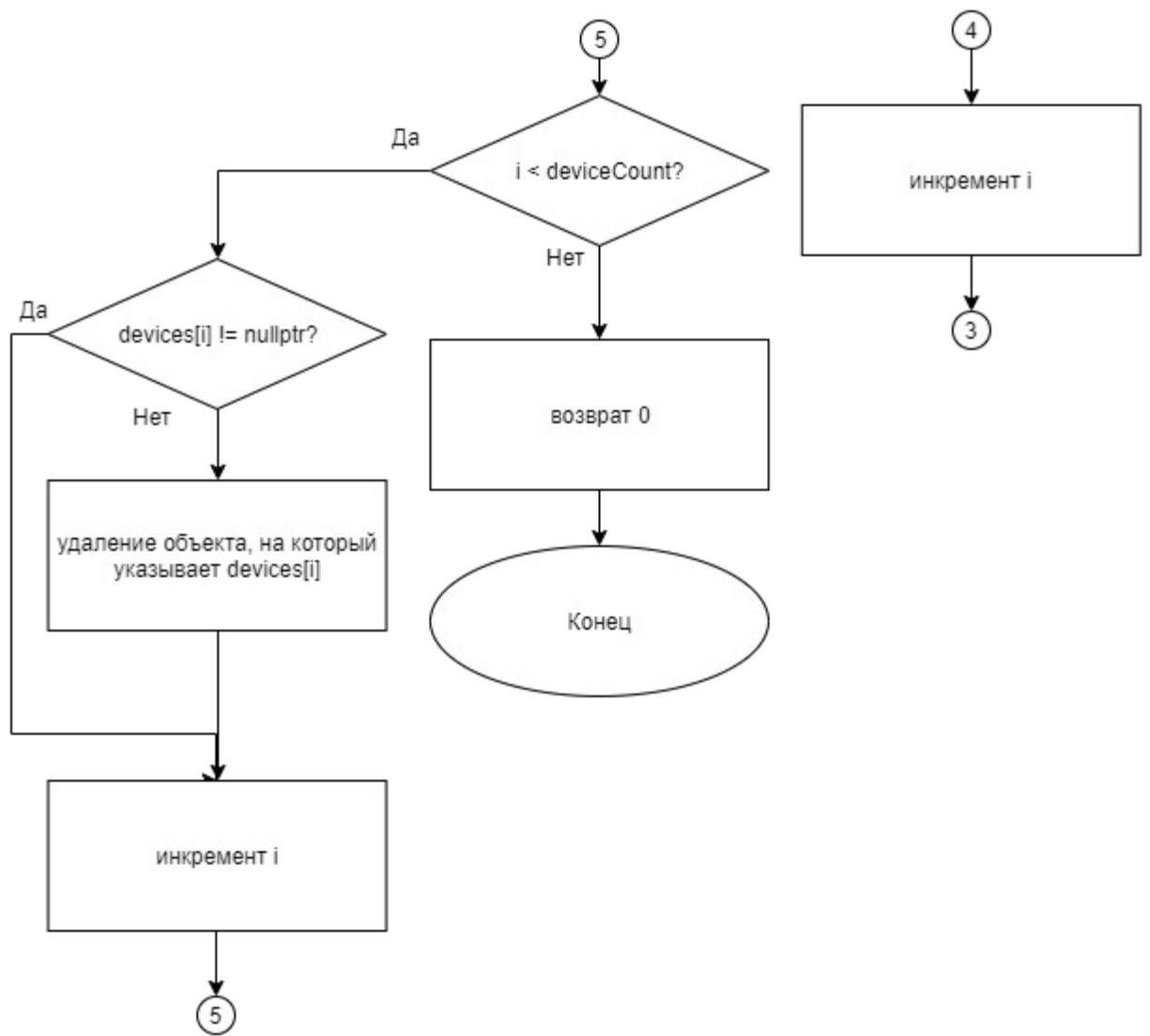


Рисунок 4 – Блок-схема алгоритма

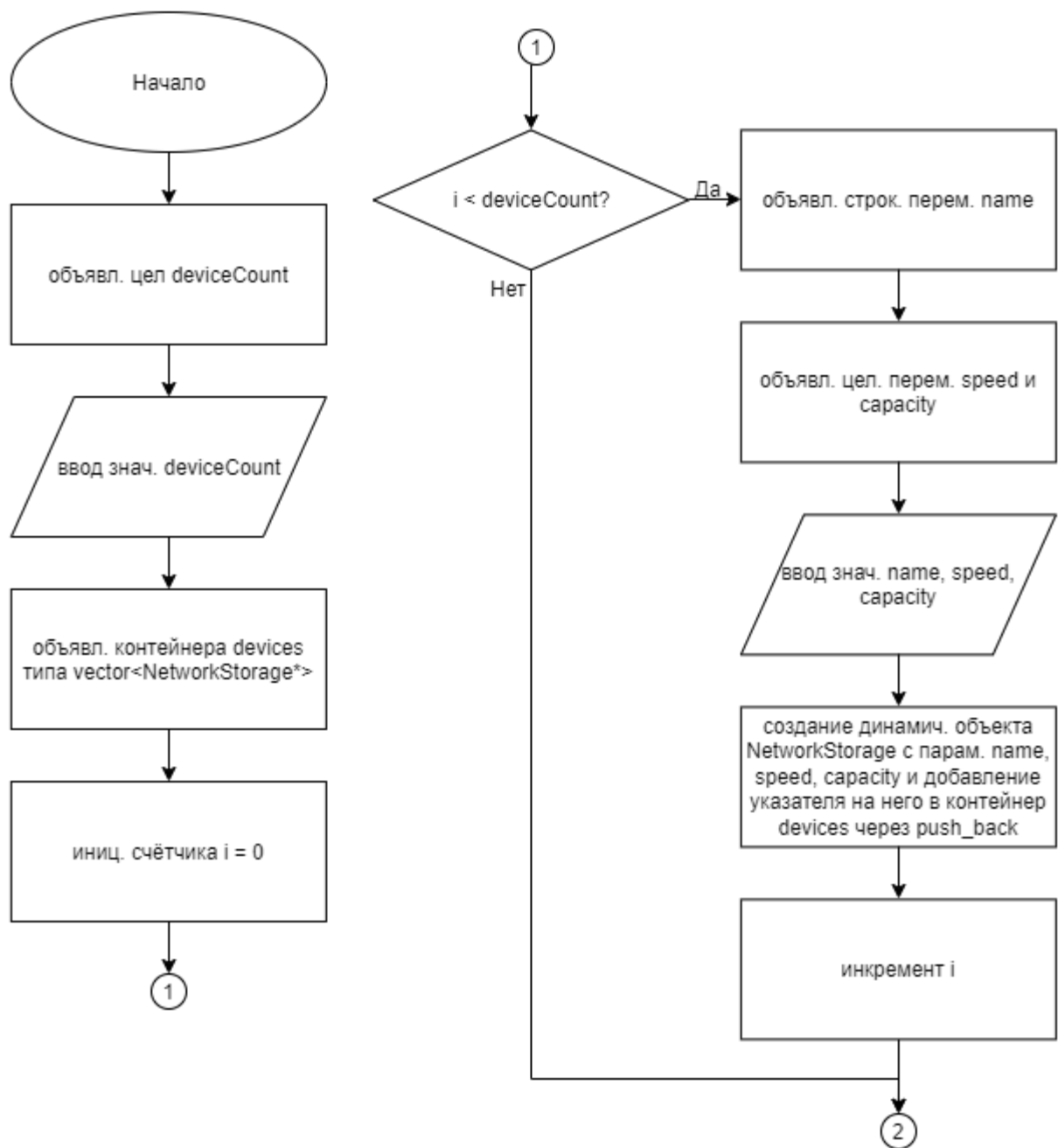


Рисунок 5 – Блок-схема алгоритма

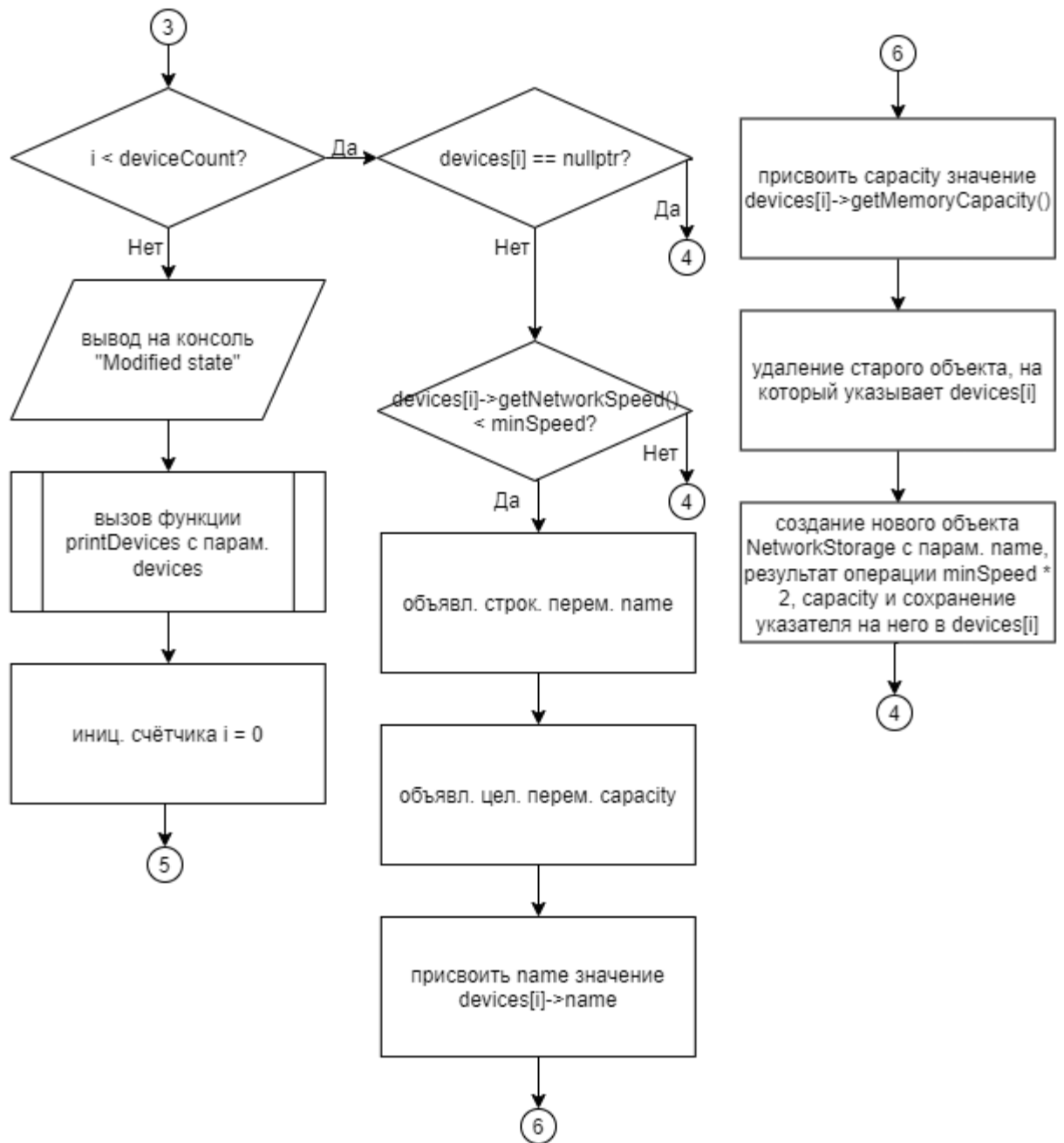


Рисунок 6 – Блок-схема алгоритма



Рисунок 7 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include <string>
#include <vector>
#include "NetworkStorage.h"
using namespace std;

void printDevices(vector<NetworkStorage*>& devices) {
    for (int i = 0; i < (int)devices.size(); i++) {
        if (devices[i] != nullptr) {
            devices[i]->printInfo();
        } else {
            cout << "Empty pointer" << endl;
        }
    }
}

int main()
{
    int deviceCount;
    cin >> deviceCount;

    vector<NetworkStorage*> devices;

    for (int i = 0; i < deviceCount; i++) {
        string name;
        int speed, capacity;
        cin >> name >> speed >> capacity;

        devices.push_back(new NetworkStorage(name, speed, capacity));
    }

    devices.push_back(nullptr);

    cout << "Initial state" << endl;
    printDevices(devices);

    int minSpeed;
```

```

        cin >> minSpeed;

        for (int i = 0; i < (int)devices.size(); i++) {
            if (devices[i] == nullptr) continue;

            if (devices[i]->getNetworkSpeed() < minSpeed) {
                string name;
                int capacity;
                name = devices[i]->name;
                capacity = devices[i]->getMemoryCapacity();

                delete devices[i];
                devices[i] = new NetworkStorage(name, minSpeed * 2, capacity);
            }
        }

        cout << "Modified state" << endl;
        printDevices(devices);

        for (int i = 0; i < (int)devices.size(); i++) {
            if (devices[i] != nullptr) {
                delete devices[i];
            }
        }

        return 0;
    }
}

```

5.2 Файл NetworkDevice.cpp

Листинг 2 – NetworkDevice.cpp

```

#include "NetworkDevice.h"

NetworkDevice::NetworkDevice(int speed) : networkSpeed(speed) {}

void NetworkDevice::setNetworkSpeed(int speed) {
    networkSpeed = speed;
}

int NetworkDevice::getNetworkSpeed() {
    return networkSpeed;
}

```

5.3 Файл NetworkDevice.h

Листинг 3 – NetworkDevice.h

```
#ifndef __NETWORKDEVICE__H
#define __NETWORKDEVICE__H

class NetworkDevice {
    int networkSpeed;

public:
    NetworkDevice(int speed);
    void setNetworkSpeed(int speed);
    int getNetworkSpeed();
};

#endif
```

5.4 Файл NetworkStorage.cpp

Листинг 4 – NetworkStorage.cpp

```
#include <iostream>
#include "NetworkStorage.h"

NetworkStorage::NetworkStorage(std::string name, int speed, int capacity)
    : NetworkDevice(speed), StorageDevice(capacity), name(name) {}

void NetworkStorage::printInfo() {
    std::cout << name << " " << getNetworkSpeed() << " " <<
    getMemoryCapacity() << std::endl;
}
```

5.5 Файл NetworkStorage.h

Листинг 5 – NetworkStorage.h

```
#ifndef __NETWORKSTORAGE__H
#define __NETWORKSTORAGE__H
#include <string>
#include "NetworkDevice.h"
#include "StorageDevice.h"
```

```

class NetworkStorage : public NetworkDevice, public StorageDevice {
public:
    std::string name;

    NetworkStorage(std::string name, int speed, int capacity);
    void printInfo();
};

#endif

```

5.6 Файл StorageDevice.cpp

Листинг 6 – StorageDevice.cpp

```

#include "StorageDevice.h"

StorageDevice::StorageDevice(int capacity) : memoryCapacity(capacity) {}

void StorageDevice::setMemoryCapacity(int capacity) {
    memoryCapacity = capacity;
}

int StorageDevice::getMemoryCapacity() {
    return memoryCapacity;
}

```

5.7 Файл StorageDevice.h

Листинг 7 – StorageDevice.h

```

#ifndef __STORAGEDEVICE__H
#define __STORAGEDEVICE__H

class StorageDevice {
    int memoryCapacity;

public:
    StorageDevice(int capacity);
    void setMemoryCapacity(int capacity);
    int getMemoryCapacity();
};

#endif

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 12.

Таблица 12 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 NAS1 100 500 NAS2 40 1000 NAS3 150 750 80	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer
3 First 0 1 Second 0 2 Third 0 3 10	Initial state First 0 1 Second 0 2 Third 0 3 Empty pointer Modified state First 20 1 Second 20 2 Third 20 3 Empty pointer	Initial state First 0 1 Second 0 2 Third 0 3 Empty pointer Modified state First 20 1 Second 20 2 Third 20 3 Empty pointer
2 First 100 0 Second 200 0 50	Initial state First 100 0 Second 200 0 Empty pointer Modified state First 100 0 Second 200 0 Empty pointer	Initial state First 100 0 Second 200 0 Empty pointer Modified state First 100 0 Second 200 0 Empty pointer

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).